DEPERTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY (INDIAN SCHOOL OF MINES) DHANBAD



Digital Image Processing Lab Manual

Subject Code: NCSH 302 L-T-P: 0-0-3

Monsoon Semester (2026-2027)

Instructor: Prof. Soumen Bag

General Lab Instructions

Do

1. Ensure the laboratory environment is well-maintained.
2. Review the theoretical background before each lab session.
3. Ask Teaching Assistants (TAs) for any clarification.
4. After completing lab assignment, you may discuss findings or issues with TAs.
5. After completion of the experiment, do the following:
    - Naming your program file as <Question No.>.<File Extension>
    - Create a single folder with a name as "<AdmissionNo.>_<AssignmentNo.>" and put all your **source files** inside of this folder.
    - Zip this folder and send through the e-mail id: lab.ncsh302@gmail.com.

Don'ts

1. Don't use the internet during the lab.
2. Don't share your codes with other students.
3. Don't disrupt the class with unnecessary noise or conversations.
4. Avoid using **mobile phones** during the lab hours.

Lab Equipment:

1. A computer that can execute Python/Matlab programs.
2. *Unix*/*Linux*/*Windows* Operating System.

Marks Distribution:

Continuous Assessment	Viva-voce	End Semester Examination
30	20	50

Lab Time: Tuesday [09:00 AM – 11:50 AM]

Lab Venue: NLHC Computer Lab-I

Content

Lab	Experiment	Page no.
Lab 1	Introduction to Matlab & Python	4
Lab 2	Basics of Image Processing	6
Lab 3	Pixel Geometry	9
Lab 4	Image Enhancement-I	12
Lab 5	Image Enhancement-II	14
Lab 6	Image Restoration	16
Lab 7	Image Morphology	19
Lab 8	Image Segmentation-I	20
Lab 9	Image Segmentation-II	21
Lab 10	Image Compression-I	22
Lab 11	Image Compression-II	25
Lab 12	Image Representation and Description	28
Lab 13	Multiresolution Analysis and Wavelets	29
Lab 14	End Semester Lab Test	30

LABORATORY 1

Introduction to Matlab and Python

Aim:

To study basic functionality of Matlab and Python.

Brief Theory:

In photography and computing, a grayscale or greyscale digital image is an image in which the value of each pixel is a single sample, that is, it carries only intensity information. Pixel is the smallest element of an image. Each pixel correspond to any one value. In an 8-bit gray scale image, the value of the pixel between 0 and 255. Images of this sort, also known as black-and-white, are composed exclusively of shades of gray, varying from black at the weakest intensity to white at the strongest.

The size of an image depends upon three things. Number of rows, Number of columns, and Number of bits per pixel (bpp). The formula for calculating the size is given below.

*Size of an image = rows * cols * bpp*

So, for a grayscale image if number of rows and columns are 1024 and 512 then

Image size = 1024 * 512 * 8 (bpp =8)

Basic Image Processing with MATLAB:

1. The Image Processing Toolbox is a collection of functions that extend the capabilities of the MATLAB's numeric computing environment. The toolbox supports a wide range of image processing operations, including: – Geometric operations – Neighborhood and block operations – Linear filtering and filter design – Transforms – Image analysis and enhancement – Binary image operations – Region of interest operations.
2. **MATLAB can import/export several image formats:** – BMP (Microsoft Windows Bitmap), GIF (Graphics Interchange Files), HDF (Hierarchical Data Format), JPEG (Joint Photographic Experts Group), PCX (Paintbrush), PNG (Portable Network Graphics), TIFF (Tagged Image File Format),XWD (X Window Dump), raw-data and other types of image data.
3. **Data types in MATLAB** – Double (64-bit double-precision floating point), Single (32-bit single-precision floating point), Int32 (32-bit signed integer), Int16 (16-bit signed integer), Int8 (8-bit signed integer), Uint32 (32-bit unsigned integer), Uint16 (16-bit unsigned integer), Uint8 (8-bit unsigned integer)
4. **Images in MATLAB** –Binary images : {0,1}, Intensity images : [0,1] or uint8, double etc., RGB images : m × n × 3, Multidimensional images: m × n × p (p is the number of layers)
5. **Image Import and Export Functions:**
   a. **Read and write images in MATLAB**

```
img = imread('apple.jpg');
dim = size(img);
figure;
imshow(img);
imwrite(img, 'output.bmp', 'bmp')
```

Basic Image Processing with Python:

1. The Image Processing Libraries, such as OpenCV (cv2), Pillow (PIL), Scikit-image (skimage), NumPy, SciPy, and Matplotlib, are collections of functions that provide image processing, computer vision, visualization, and numerical computing capabilities in Python. These libraries support a wide range of image processing operations, including: – Image input/output (I/O) – Image display and visualization – Color space conversion – Geometric transformations (resize, rotate, crop, flip) – Neighborhood and block operations – Point operations – Linear and nonlinear filtering – Filter design – Image transforms (Fourier, Wavelet, DCT) – Image enhancement and restoration – Image arithmetic and logical operations – Thresholding and binary image operations – Morphological operations – Edge, line, and corner detection – Image segmentation – Feature detection and extraction – Region of Interest (ROI) operations – Image analysis and measurement – Image compression – Object detection and recognition.
2. **Python can import and export several image formats** through libraries such as OpenCV, Pillow (PIL), and Scikit-image, including: – BMP (Microsoft Windows Bitmap) – JPEG/JPG (Joint Photographic Experts Group) – PNG (Portable Network Graphics) – TIFF (Tagged Image File Format) – GIF (Graphics Interchange Format) – WebP (Web Picture Format) – and many other commonly used image formats.
3. **Data types in Python (NumPy**) – bool (Boolean), float64 (64-bit double-precision floating-point), float32 (32-bit single-precision floating-point), int8 (8-bit signed integer), int16 (16-bit signed integer), int32 (32-bit signed integer), int64 (64-bit signed integer), uint8 (8-bit unsigned integer), uint16 (16-bit unsigned integer), uint32 (32-bit unsigned integer), and uint64 (64-bit unsigned integer).
4. **Images in Python (NumPy)** – Binary images: bool or {0,1}, Grayscale (Intensity) images: uint8 (0–255), uint16, float32, or float64, RGB images: m × n × 3, Multidimensional images: m × n × p (where p is the number of channels or layers).
5. **Image Import and Export Functions:**
   a. **Read and write images in Python**

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('apple.jpg')
dim = img.shape
plt.figure()
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
cv2.imwrite('output.bmp', img)
plt.show()
```

Observations:

- Students will learn how to import and export digital images.
- Student will learn how perform various label of image conversations.

Questions:

1. Problems based on image loading an image.
2. Problems based on showing an image.

LABORATORY 2

Basics of Image Processing

Aim:

To study how to write a gray and color image, how to convert images into various color model etc.

Brief Theory:

Digital images play an important role both in daily life applications as well as in the areas of research technology. The digital image processing refers to the manipulation of an image by means of processor. The different elements of an image processing system include image acquisition, image storage, image processing and display. An image is two dimensional function that represent a message of sum characteristics such as brightness or color of viewed scene. An image is defined by the mathematical function f(x,y) where x and y are the two co-ordinates horizontally and vertically. The value of f(x,y) at any point is gives the pixel value at that point of an image.

Basic Image Processing with MATLAB:

1. **Image Display**:
   - **a.** image - create and display image object
   - **b.** imagesc - scale and display as image
   - **c.** imshow - display image
   - **d.** colorbar- display colorbar
   - **e.** getimage- get image data from axes
   - **f.** truesize- adjust display size of image
   - **g.** zoom - zoom in and zoom out of 2D plot
   - **h.** pixval on- turns on the pixel values in our figure
   - **i.** impixel(i,j)- the command returns the value of the pixel (i,j)
   - **j.** iminfo- information about the image.

2. **Image Conversion**:
   - **a.** gray2ind - intensity image to index image
   - **b.** im2bw - image to binary
   - **c.** im2double - image to double precision
   - **d.** im2uint8- image to 8-bit unsigned integers
   - **e.** im2uint16 - image to 16-bit unsigned integers
   - **f.** ind2gray- indexed image to intensity image
   - **g.** mat2gray - matrix to intensity image
   - **h.** rgb2gray- RGB image to grayscale
   - **i.** rgb2ind - RGB image to indexed image
3. **Arithmetic Operations**:
   - **a.** These operations act by applying a simple function y=f(x) to each gray value in the image.
   - **b.** Simple functions include adding or subtract a constant value to each pixel: $y = x \pm C$ (imadd, imsubtract)
   - **c.** Multiplying each pixel by a constant: $y = C \cdot x$ (immultiply, imdivide)

d. Complement: For a grayscale image, it is a photographic negative.

Basic Image Processing with Python:

1. **Image Display**
   - **a.** cv2.imread() – Read an image from a file.
   - **b.** plt.imshow() – Display an image.
   - **c.** plt.figure() – Create a new figure window.
   - **d.** plt.title() – Display a title for the image.
   - **e.** plt.colorbar() – Display a colorbar for an image.
   - **f.** img.shape – Get the dimensions of an image.
   - **g.** img.size – Get the total number of pixels in an image.
   - **h.** img.dtype – Display the data type of the image.
   - **i.** plt.axis() – Show or hide image axes.
   - **j.** plt.xlim(), plt.ylim() – Zoom into a specific region of an image.
   - **k.** plt.ginput() – Select pixel coordinates interactively from an image.
   - **l.** img[i, j] – Return the pixel value at location (i, j).
   - **m.** cv2.imwrite() – Save an image to a file.

2. **Image Conversion**:
   - **a.** cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) – Convert a BGR image to a grayscale image.
   - **b.** cv2.cvtColor(img, cv2.COLOR_BGR2RGB) – Convert a BGR image to an RGB image.
   - **c.** cv2.cvtColor(img, cv2.COLOR_GRAY2BGR) – Convert a grayscale image to a BGR image.
   - **d.** cv2.threshold() – Convert a grayscale image to a binary image.
   - **e.** img.astype(np.float32) – Convert an image to 32-bit floating-point.
   - **f.** img.astype(np.float64) – Convert an image to 64-bit double-precision floating-point.
   - **g.** img.astype(np.uint8) – Convert an image to 8-bit unsigned integers.
   - **h.** img.astype(np.uint16) – Convert an image to 16-bit unsigned integers.
   - **i.** cv2.normalize() – Normalize image intensity values.
   - **j.** cv2.applyColorMap() – Apply a color map to a grayscale image.

3. **Arithmetic Operations**:
   - **a.** These operations apply a mathematical function $y = f(x)$ to each pixel value in the image.
   - **b.** Addition/Subtraction: Add or subtract a constant value from each pixel:
     $y = x \pm C$ (cv2.add(), cv2.subtract())
   - **c.** Multiplication/Division: Multiply or divide each pixel by a constant:
     $y = C \cdot x$ (cv2.multiply(), cv2.divide())
   - **d.** Absolute Difference: Compute the absolute difference between two images:
     (cv2.absdiff())
   - **e.** Bitwise Operations: Perform AND, OR, XOR, and NOT operations on images:
     (cv2.bitwise_and(), cv2.bitwise_or(), cv2.bitwise_xor(), cv2.bitwise_not())
   - **f.** Image Complement (Negative): Compute the negative of a grayscale image:
     $y = 255 - x$ (cv2.bitwise_not())

Observations:

- Students will learn how to import and export digital images.
- Student will learn how perform various types of image conversations.
- Students will learn different arithmetic operations is required for recovery and modify an image.

Questions:

1. Problems based on image write, resize and conversation etc.
2. Problems based on practical use of image arithmetic operations.

LABORATORY 3

Pixel Geometry

Aim:

Study the pixel representation of an image and the relationship between pixels.

Brief Theory:

Image Interpolation:
Used in image resizing (zooming and shrinking), rotating, and geometric corrections. Interpolation is the process of using known data to estimate values at unknown locations.

Nearest Neighbor interpolation: It assigns to each new location the intensity of its nearest neighbor in the original image. Produce undesirable artefacts, such as severe distortion of straight edges.

Bilinear Interpolation: We use the four nearest neighbors to estimate the intensity.

$$V(x, y) = ax + by + cxy + d$$

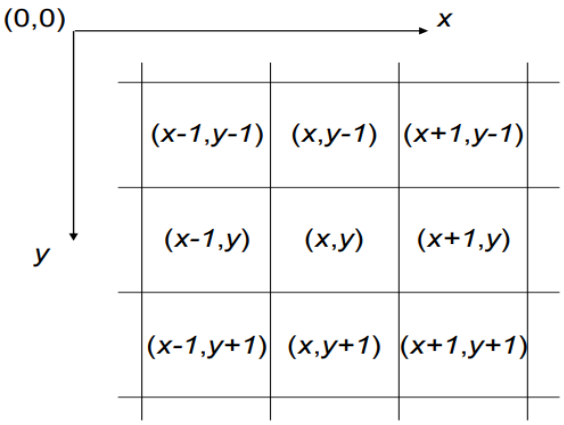
Need to solve four equations. Better results than nearest neighbor interpolation, with a modest increase in computational burden.

Bicubic Interpolation: Involves sixteen neighbors to estimate intensity.

$$V(x, y) = \sum\sum a_{ij} x_i \, y_j$$

Need to solve sixteen equations. Gives better results than other methods. More complex. Used in Adobe Photoshop, and Corel Photopaint.

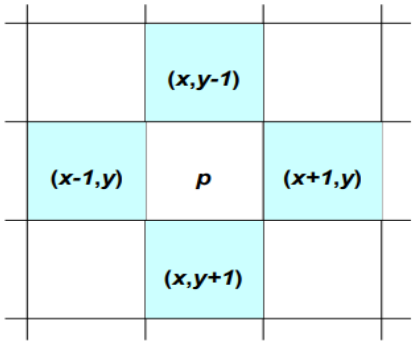
Basic Relationship of Pixels:



Conventional indexing method

Neighbors of a Pixel:

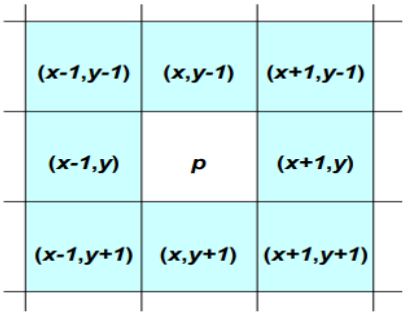
Neighborhood relation is used to tell adjacent pixels. It is useful for analyzing regions.



4-neighbors of *p*:

$$N_4(p) = \{(x-1,y),\ (x+1,y),\ (x,y-1),\ (x,y+1)\}$$

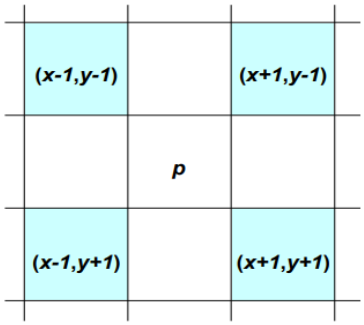
4-neighborhood relation considers only vertical and horizontal neighbors.



8-neighbors of *p*:

$$N_8(p) = \{(x-1,y-1),\ (x,y-1),\ (x+1,y-1),\ (x-1,y),\ (x+1,y),\ (x-1,y+1),\ (x,y+1),\ (x+1,y+1)\}$$

8-neighborhood relation considers all neighbor pixels.



Diagonal neighbors of *p*:

$$N_D(p) = \{(x-1,y-1),\ (x+1,y-1),\ (x-1,y+1),\ (x+1,y+1)\}$$

Diagonal -neighborhood relation considers only diagonal neighbor pixels.

Connectivity:
Connectivity is adapted from neighbourhood relation. Two pixels are connected if they are in the same class (i.e. the same color or the same range of intensity) and they are neighbors of one another. For p and q from the same class

- 4-connectivity: p and q are 4-connected $if\ q \in N4\ (p)$
- 8-connectivity: p and q are 8-connected $if\ q \in N8\ (p)$
- Mixed-connectivity (m-connectivity): p and q are m-connected

$$if\ q \in N4\ (p)\ or\ q \in ND\ (p)\ and\ N4\ (p) \cap N4\ (q) = \varnothing$$

Adjacency: A pixel p is adjacent to pixel q is they are connected. Two image subsets $S1$ and $S2$ are adjacent if some pixel in $S1$ is adjacent to some pixel in $S2$.

Path: A path from pixel p at (x, y) to pixel q at (s, t) is a sequence of distinct pixels: $(x_0, y_0), (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ such that $(x_0, y_0) = (x, y)$ and $(x_n, y_n) = (s, t)$ and (xi, yi) is adjacent to $(x_i - 1, y_i - 1), i = 1, \dots, n$. We can define type of path: 4-path, 8-path or m-path depending on type of adjacency.

Distance: For pixel p, q, and z with coordinates $(x, y), (s, t)$ and (u, v), D is a distance function or metric if

- $D(p, q) \geq 0$ ($D(p, q) = 0$ if and only if $p = q$
- $D(p, q) = D(q, p)\ w\ D(p, z) \leq D(p, q) + D(q, z)$
- Example: Euclidean distance, city-block distance, and chessboard distance.

<u>Observations</u>:

- Observe the behavior of pixels and the relationship between them.
- Learn how path, relative position of pixels helps in calculation higher degree of application of geometry.

<u>Questions</u>:

1. Implement the different distance strategy to find the relativeness of pixels.
2. Check the adjacent position of pixels.
3. Compute the different interpolation techniques for gray scale image.

LABORATORY 4

Image Enhancement – I

Aim:
To study and implement image enhancement techniques in the spatial domain — gray-level transformations, histogram equalization, and spatial filtering (smoothing and sharpening) — using MATLAB/Python, and to observe their effect on image quality.

Brief Theory:
Image enhancement is the process of making an image more useful by highlighting interesting detail, removing noise, or making the image more visually appealing. Enhancement techniques used for this purpose are broadly divided into two categories: spatial domain techniques, which work by direct manipulation of the pixel values of an image, and frequency domain techniques, which work by manipulating the Fourier (or wavelet) transform of an image.

Gray-Level Transformations: A general spatial domain enhancement process can be written as g(x, y) = T[f(x, y)], where f(x, y) is the input image, g(x, y) is the output image, and T is an operator defined over a neighbourhood of (x, y). When the neighbourhood is a single pixel, T becomes a point (gray-level) transformation such as the image negative, s = (L − 1) − r, which enhances white/gray detail embedded in dark regions; the log transformation, s = c·log(1 + r), which expands dark values and compresses bright values; and the power-law (gamma) transformation, s = c·r^γ, where γ < 1 brightens dark regions and γ > 1 darkens the image.

Histogram and Histogram Equalization: The histogram of a digital image with gray levels in the range [0, L − 1] is a discrete function h(rk) = nk, where rk is the k-th gray level and nk is the number of pixels having that level. Histogram equalization is a technique that redistributes the gray levels of an image so that the output histogram is approximately uniform, using the transformation sk = (L − 1) Σ(pr(rj)), summed over j = 0 to k, where pr(rj) = nj / N and N is the total number of pixels. This spreads out the most frequent intensity values and improves the global contrast of the image.
Spatial Filtering: Spatial filters use a small neighbourhood (mask) that is moved over every pixel of the image; the output pixel value is computed from the pixel values under the mask. Spatial filters are broadly classified as smoothing filters and sharpening filters.

Smoothing Spatial Filters: these are used for blurring and for noise reduction. Types of smoothing filters include:

- **Averaging (Mean) Filter:** replaces the value of every pixel by the average of the gray levels in the neighbourhood defined by the filter mask; all coefficients are equal, and it is effective at reducing noise but blurs edges.
- **Weighted Averaging Filter:** similar to the averaging filter, but pixels are multiplied by different coefficients, with the center pixel weighted more heavily than the average filter.

• **Median Filter:** an order-statistics (non-linear) filter that replaces each pixel with the median of the gray levels in its neighbourhood. It is particularly effective at removing salt-and-pepper noise while preserving edges better than the mean filter.

Sharpening Spatial Filters: these are also known as derivative filters, and their purpose is the opposite of smoothing filters — the removal of blurring and highlighting of edges and fine detail. The simplest and most commonly used sharpening filter is the Laplacian filter, a second-derivative operator defined (for one common mask) as:

$\nabla^2 f = [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] - 4f(x, y)$

Applying the Laplacian mask to an image produces a Laplacian image that highlights intensity discontinuities and suppresses regions of slowly varying gray level. The final sharpened image is obtained by combining the Laplacian image with the original image: $g(x, y) = f(x, y) + c \cdot \nabla^2 f(x, y)$, where c depends on the sign convention of the mask used. This process is also referred to as unsharp masking when the roles of blurred and original images are used to derive the same effect.

Observations:

- Students will learn how image negative, log, and power-law (gamma) transformations affect image brightness and contrast.
- Students will learn how histogram equalization improves the global contrast of a low-contrast image.
- Students will learn the difference between mean and median filtering for noise removal, and why the median filter better preserves edges.
- Students will learn how the Laplacian filter highlights edges and fine detail, and how it is used to sharpen an image.

Questions:

1. Write a program to read a grayscale image and obtain its negative, log-transformed, and gamma-corrected (for at least two values of γ) versions.
2. Write a program to perform histogram equalization on a low-contrast image and plot the histograms before and after equalization.
3. Write a program to add salt-and-pepper noise to an image and compare the results of averaging (mean) and median filtering of sizes 3×3 and 5×5.
4. Write a program to sharpen an image using the Laplacian filter and display the original, Laplacian, and sharpened images.

LABORATORY 5

Image Enhancement – II

Aim:

To study and implement image enhancement techniques in the frequency domain using the 2-D Discrete Fourier Transform (DFT), and to design and apply Ideal, Butterworth, and Gaussian low-pass and high-pass filters using MATLAB/Python.

Brief Theory:

Frequency domain enhancement techniques operate on the Fourier transform of an image rather than on its pixel values directly. Any two-dimensional image f(x, y) of size M × N can be represented in the frequency domain using the 2-D Discrete Fourier Transform (DFT):

$F(u, v) = \Sigma\Sigma\ f(x, y) \cdot \exp[-j2\pi(ux/M + vy/N)]$, summed over x = 0..M−1 and y = 0..N−1.

Low frequencies in F(u, v) correspond to slowly varying intensity components (smooth regions of the image), while high frequencies correspond to fine detail, edges, and noise. The general steps of frequency domain filtering are: (i) multiply the input image by (−1)^(x+y), or equivalently apply an FFT-shift, to center the transform; (ii) compute the 2-D DFT, F(u, v); (iii) multiply F(u, v) by a filter transfer function H(u, v) to obtain G(u, v) = H(u, v)·F(u, v); (iv) compute the inverse DFT of G(u, v); and (v) take the real part and undo the centering step to obtain the enhanced image g(x, y).

Low-Pass Filters (Smoothing): a low-pass filter attenuates high frequencies while preserving low frequencies, producing a smoothed image. Let D(u, v) denote the distance of a point (u, v) from the centre of the frequency rectangle, and D0 the cut-off distance. Three common low-pass filters are:

- **Ideal Low-Pass Filter (ILPF):** H(u, v) = 1 if D(u, v) ≤ D0, and 0 otherwise. It produces ringing artefacts in the output image because of its sharp cut-off.
- Butterworth Low-Pass Filter (BLPF): H(u, v) = 1 / [1 + (D(u, v)/D0)^(2n)], where n is the filter order. It provides a smooth transition between passed and attenuated frequencies, with negligible ringing for low values of n.
- **Gaussian Low-Pass Filter (GLPF):** $H(u, v) = \exp[-D(u, v)^2 / (2D0^2)]$. It provides the smoothest transition of the three and produces no ringing artefacts.

High-Pass Filters (Sharpening): a high-pass filter attenuates low frequencies while preserving high frequencies, sharpening the image by emphasising edges and fine detail. A high-pass filter transfer function is obtained from its low-pass counterpart as H(u, v) = 1 − H'(u, v), where H'(u, v) is the transform of the corresponding low-pass filter. This gives the Ideal (IHPF), Butterworth (BHPF), and Gaussian (GHPF) high-pass filters. Since the high-pass filter removes the DC (average intensity) term, the resulting image often appears dark, with bright edges.

Band-Pass and Band-Reject Filters: a band-pass filter passes a specific band of frequencies while attenuating frequencies outside that band, and is useful for enhancing edges while simultaneously reducing noise; a band-reject filter does the opposite, attenuating a specific band of frequencies (commonly used to remove periodic noise).

Observations:

- Students will learn how to compute and visualize the 2-D DFT (magnitude spectrum) of an image.
- Students will learn the effect of low-pass filtering (Ideal, Butterworth, Gaussian) on image smoothing, and why the Ideal filter produces ringing while the Gaussian filter does not.
- Students will learn how high-pass filtering in the frequency domain sharpens an image by preserving edges and attenuating smooth background regions.
- Students will learn why frequency domain analysis is useful in image processing, and the relationship between spatial convolution and frequency-domain multiplication.

Questions:

1. Write a program to compute and display the 2-D DFT magnitude spectrum of a grayscale image.
2. Write a program to design and apply Ideal, Butterworth, and Gaussian low-pass filters on an image for a given cut-off frequency D0, and compare the outputs.
3. Write a program to design and apply Ideal, Butterworth, and Gaussian high-pass filters on an image, and compare the sharpening effect of each.

LABORATORY 6

Image Restoration

Aim:

To study the image degradation/restoration model, various noise models, and spatial and frequency domain filtering techniques used to restore an image degraded by noise and/or blur.

Brief Theory:

Image restoration refers to the process of recovering an image that has been degraded, by using a-priori knowledge of the degradation phenomenon. Restoration techniques are oriented toward modelling the degradation and applying the inverse process in order to recover the original image, unlike enhancement which is largely a subjective process. Restoration is an objective process in the sense that a degradation model can often be formulated and used to arrive at a restored image.

Degradation Model: A degraded image can be modelled as $g(x,y) = h(x,y) * f(x,y) + n(x,y)$, where $f(x,y)$ is the original image, $h(x,y)$ is the degradation function (e.g. blur due to motion or defocus), $n(x,y)$ is additive noise, and * denotes convolution. When h is a simple identity (no blur), restoration reduces to noise removal, which can be handled entirely using spatial domain filters.

Noise Models: Noise in digital images generally arises during image acquisition and/or transmission. The most important noise probability density functions used to model noise are Gaussian, Rayleigh, Erlang (Gamma), Exponential, Uniform, and Impulse (salt-and-pepper) noise. The parameters of the noise (mean, variance) can be estimated by examining the histogram of a small, flat, constant-intensity patch of a captured image.

Restoration in the Presence of Noise Only – Spatial Filtering: When the only degradation present is noise (i.e., $g(x,y) = f(x,y) + n(x,y)$), spatial filtering is the method of choice to reduce its effect.

(a) Mean Filters: These filters compute a value based on a neighbourhood window; the four common ones are:

1. Arithmetic mean filter – averages the pixels in the neighbourhood.
2. Geometric mean filter – the Nth root of the product of the pixel values in the neighbourhood; achieves smoothing comparable to the arithmetic mean filter but tends to lose less image detail.
3. Harmonic mean filter – works well for salt noise and Gaussian noise, but fails for pepper noise.
4. Contraharmonic mean filter – well suited for reducing or virtually eliminating salt-and-pepper noise; its order Q is positive for pepper noise and negative for salt noise.

(b) Order-Statistics Filters: These filters are based on the ordering (ranking) of the pixels contained in the neighbourhood:1. Median filter – very effective for bipolar and unipolar impulse (salt-and-pepper) noise.

2. Max and Min filters – useful for finding the brightest/darkest points and for reducing pepper/salt noise respectively.
3. Midpoint filter – averages the maximum and minimum values in the neighbourhood; works best for randomly distributed noise such as Gaussian or uniform noise.
4. Alpha-trimmed mean filter – deletes the d/2 lowest and d/2 highest grey-level values in the neighbourhood and averages the remaining ones; useful when multiple types of noise are present together, such as a combination of salt-and-pepper and Gaussian noise.

(c) Adaptive Filters: An adaptive filter changes its behaviour based on the statistical characteristics of the image inside the filter region. The adaptive local noise reduction filter uses the local mean and variance inside a neighbourhood, while the adaptive median filter can handle impulse noise with probabilities larger than a simple median filter can, and also performs some smoothing for non-impulse noise, while preserving edges by varying the filter window size.

Periodic Noise Reduction by Frequency Domain Filtering: Periodic noise in an image arises typically from electrical or electromechanical interference during image acquisition and appears as concentrated bursts of energy in the Fourier domain. This type of noise can be reduced using frequency domain filters such as bandreject filters, bandpass filters, and notch filters (which reject/pass a small region around a specific frequency). Optimum notch filtering further minimises local variance of the restored image to improve results.

Restoration in the Presence of Noise and Blur: When both blur and noise are present, restoration requires knowledge (or an estimate) of the degradation function H(u,v) in the frequency domain.

1. Inverse filtering – $F'(u,v) = G(u,v)/H(u,v)$; the simplest approach but highly sensitive to noise, especially where H(u,v) is small.
2. Wiener filtering (minimum mean square error filtering) – incorporates the degradation function and statistical characteristics of noise into the restoration process, minimising the mean square error between the estimated and original image.
3. Constrained least squares filtering – requires only the mean and variance of the noise, rather than its full power spectrum, and imposes a smoothness constraint on the restored image.
4. Geometric mean filter – a generalized formulation that can be tuned (via its parameters) to behave like an inverse filter, a Wiener filter, or something in between.

Observations:

- Students will understand the image degradation/restoration model and the different noise probability density functions that affect images.
- Students will implement mean filters and order-statistics filters for restoring images degraded purely by noise.
- Students will study adaptive filters and frequency domain restoration techniques (notch filtering, inverse filtering, and Wiener filtering) for images degraded by both blur and noise.

Questions:

1. Problems based on adding different types of noise (Gaussian, salt-and-pepper, uniform) to an image and restoring it using mean and order-statistics filters.
2. Problems based on periodic noise removal using bandreject and notch filtering in the frequency domain.
3. Problems based on restoring a blurred and noisy image using inverse filtering and Wiener filtering, and comparing the results.

LABORATORY 7

Image Morphology

Aim:

To study the several morphological algorithms.

Brief Theory:

Morphology is a branch of biology that deals with form and structure of animal and plant. In image processing, we use mathematical morphology to extract image components which are useful in representation and description of region shape such as Boundaries, Skeletons, Convex hull, Thinning, Pruning etc. Morphological operations rely only on the relative ordering of pixel values, not on their numerical values, and therefore are especially suited to the processing of binary images. Morphological operations can also be applied to greyscale images such that their light transfer functions are unknown and therefore their absolute pixel values are of no or minor interest. Two Fundamental morphological operations are: Erosion and Dilation.

Erosion and dilation are two fundamental image morphological operations. Dilation adds pixels to the boundaries of objects in an image, while erosion removes pixels on object boundaries.

Dilation operation: The value of the output pixel is the maximum value of all the pixels in the input pixel's neighborhood. In a binary image, if any of the pixels is set to the value 1, the output pixel is set to 1.

Erosion operation: The value of the output pixel is the minimum value of all the pixels in the input pixel's neighborhood. In a binary image, if any of the pixels is set to 0, the output pixel is set to 0.

Opening and closing operations can be done by combination of erosion and dilation in different sequence.

Observations:

- Morphological image processing is a collection of non-linear operations related to the shape or morphology of features in an image.
- Pursues the goals of removing binary image's numerous imperfections by accounting for the form and structure of the image.

Questions:

1. Program to demonstrate Erosion and Dilation.
2. Write a program for edge detection using different edge detection mask.
3. Write and execute program for image morphological operations erosion and dilation.
4. Implement Morphological smoothing using opening and closing

LABORATORY 8

Image Segmentation I

Aim:

To study various image segmentation techniques.

Brief Theory:

Image segmentation is the division of an image into regions or categories, which correspond to different objects or parts of objects. Every pixel in an image is allocated to one of a number of these categories. A good segmentation is typically one in which possesses following characteristics:

Pixels in the same category have similar grayscale of multivariate values and form a connected region.

Neighbouring pixels which are in different categories have dissimilar values.

Segmentation is often the critical step in image analysis: the point at which we move from considering each pixel as a unit of observation to working with objects (or parts of objects) in the image, composed of many pixels. If segmentation is done well then all other stages in image analysis are made simpler. There are three general approaches to segmentation, namely thresholding, edge-based methods, and region-based methods.

In **thresholding**, pixels are allocated to categories according to the range of values in which a pixel lies. Pixels with values less than a particular threshold value are placed in one category, and remaining pixels are placed in the other category.

In **edge-based segmentation**, an edge filter is applied to the image, pixels are classified as edge or non-edge depending on the filter output, and pixels which are not separated by an edge are allocated to the same category.

Region-based segmentation algorithms operate iteratively by grouping together pixels which are neighbours and have similar values and splitting groups of pixels which are dissimilar in value.

Observations:

- The students will explore different basic image segmentation techniques.
- The students will study the applicability of basic image segmentation methods into a number of application domains.

Questions:

1. Problems based on thresholding-based segmentation.
2. Problems based on edge-based segmentation.
3. Problems based on Region-based segmentation.

LABRATORY 9

Image Segmentation II

Aim:

To study point and line detection using Hough transform.

Brief Theory:

In automated analysis of digital images, a sub problem often arises of detecting simple shapes, such as straight lines, circles or ellipses. In many cases an edge detector can be used as a pre-processing stage to obtain image points or image pixels that are on the desired curve in the image space. Due to imperfections in either the image data or the edge detector, however, there may be missing points or pixels on the desired curves as well as spatial deviations between the ideal line/circle/ellipse and the noisy edge points as they are obtained from the edge detector. For these reasons, it is often non-trivial to group the extracted edge features to an appropriate set of lines, circles or ellipses. The purpose of the Hough transform is to address this problem by making it possible to perform groupings of edge points into object candidates by performing an explicit voting procedure over a set of parameterized image objects.

The simplest case of *Hough transform* is detecting straight lines. In general, the straight line $y = mx + b$ can be represented as a point (b, m) in the parameter space. However, vertical lines pose a problem. They would give rise to unbounded values of the slope parameter m. Thus, for computational reasons, Hesse normal form is used as follows

$$r = x \cos \theta + y \sin \theta$$

where r is the distance from the origin to the closest point on the straight line, and θ is the angle between the x axis and the line connecting the origin with that closest point. It is therefore possible to associate with each line of the image a pair (r, θ). The (r, θ) plane is sometimes referred to as Hough space for the set of straight lines in two dimensions. This representation makes the Hough transform conceptually very close to the two-dimensional Radon transform. In fact, the Hough transform is mathematically equivalent to the Radon transform, but the two transformations have different computational interpretations traditionally associated with them. Given a single point in the plane, then the set of all straight lines going through that point corresponds to a sinusoidal curve in the (r, θ) plane, which is unique to that point. A set of two or more points that form a straight line will produce sinusoids which cross at the (r, θ) for that line. Thus, the problem of detecting collinear points can be converted to the problem of finding concurrent curves.

Observations:

- The students will understand Haugh transform for line detection.
- The students will study the applicability of Haugh transform for different applications.

Questions:

1. Problems based on line detection.
2. Problems based on application of Haugh transform.

LABORATORY 10

Image Compression – I

Aim:

To study lossless image compression techniques – prefix and uniquely decodable codes, Shannon-Fano coding, Huffman coding, and arithmetic coding – and to compute code efficiency and compression ratio for a given image histogram.

Brief Theory:

Image compression is defined as the process of reducing the amount of data needed to represent a digital image, by removing redundant data, so as to decrease the number of bits required to store and transmit an image. Two types of digital image compression are lossless and lossy compression; lossless compression allows the original data to be recovered exactly from the compressed data, whereas lossy compression allows some controlled loss of information in exchange for a much higher compression ratio.

Uniquely Decodable Codes

A variable length code assigns a bit string (codeword) of variable length to every message value, e.g. a = 1, b = 01, c = 101, d = 011. Given a bit sequence such as 1011, it is not immediately clear whether it represents aba, ca, or ad. A uniquely decodable code is a variable length code in which bit strings can always be uniquely decomposed into their constituent codewords.

Prefix Codes

A prefix code is a variable length code in which no codeword is a prefix of another codeword. This makes it possible to decode each message in an encoded string as soon as its last bit is received, without reference to succeeding code symbols – i.e., decoding can be performed instantaneously using a simple binary decoding tree. The Huffman code is an example of a prefix code.

Shannon-Fano Coding

Shannon-Fano coding is a top-down binary tree method that does not, in general, give an optimal variable length code. The algorithm proceeds as follows:

1. The set of source symbols is sorted in the order of non-increasing probabilities.
2. Each source symbol is interpreted as the root of a tree.
3. The list is divided into two groups such that each group has nearly equal total probability.
4. The codeword of the first group is appended with 0.
5. The codeword of the second group is appended with 1.
6. Steps 3–5 are repeated for each group until each group contains only one symbol.

Huffman Coding

The Huffman code, developed by D. Huffman in 1952, is a minimum length code: given the statistical distribution of the gray levels (the histogram), the Huffman algorithm generates a code that is as close as possible to the minimum bound, the entropy. It results in an unequal (variable) length code, and for complex images typically reduces the file size by 10% to 50% (a compression ratio of 1.1:1 to 1.5:1), improvable to 2:1 or 3:1 by preprocessing to remove irrelevant information.

The binary Huffman algorithm proceeds as follows:

1. Find the gray level probabilities for the image from its histogram.

2. Order the input probabilities (histogram magnitudes) from smallest to largest.

3. Combine the smallest two probabilities by addition, reordering the list if necessary.

4. Repeat step 3 until only two probabilities remain.

5. Working backward along the resulting tree, generate the code by alternately assigning 0 and 1 at each branch.

Example: Suppose an image with 2 bits/pixel (four gray levels), 10 rows by 10 columns, has the histogram: gray level 0 has 20 pixels, level 1 has 30 pixels, level 2 has 10 pixels, and level 3 has 40 pixels, giving probabilities g0 = 0.2, g1 = 0.3, g2 = 0.1, g3 = 0.4. Ordering and combining the two smallest probabilities repeatedly (0.1 + 0.2 = 0.3, then 0.3 + 0.3 = 0.6, then 0.6 + 0.4 = 1.0) and assigning 0/1 backward along the tree gives the Huffman code: g0 = 010, g1 = 00, g2 = 011, g3 = 1.

The entropy of this source is Entropy = $-\Sigma$ pi log2(pi) ≈ 1.846 bits/pixel, while the average Huffman codeword length is Lave = Σ li pi = 3(0.2) + 2(0.3) + 3(0.1) + 1(0.4) = 1.9 bits/pixel. The Huffman code efficiency is therefore Entropy / Lave = 1.846/1.9 ≈ 0.9716, i.e. 97.16%, giving a compression of about 2.0 : 1.9 (a ratio of about 1.05, i.e. roughly 5% compression). Since Huffman coding is highly dependent on the histogram, any preprocessing that simplifies the histogram will improve the compression ratio.

Arithmetic Coding

Huffman coding assigns a fixed-length codeword (an integer number of bits, minimum one bit) to each symbol individually, irrespective of context. This can be far from optimal: for a data stream of two symbols 'a' and 'b' with probabilities 0.90 and 0.10 respectively, the entropy is only about 0.47 bits/symbol, but Huffman coding must still assign at least 1 bit per symbol – more than double the theoretical requirement. Huffman coding is also non-adaptive: the data must first be scanned to build a fixed codeword table, which then remains unchanged throughout coding.

Arithmetic coding overcomes these limitations by transforming the entire input sequence into a single floating point ("tag") number between 0 and 1, based on the cumulative probability distribution of the symbols, rather than assigning an independent code to each symbol. There is no direct correspondence between the code and individual pixel values, no codebook needs to be stored (only the source statistics), and the interval [0, 1) is successively subdivided according to the placement of each successive symbol in the probability distribution. As images are large and computer precision is finite, the image must in practice be divided into smaller sub-images before encoding.

Encoding: A source emits four symbols {a, b, c, d} with probabilities 0.4, 0.2, 0.1, and 0.3 respectively, giving the sub-ranges a: (0.0–0.4), b: (0.4–0.6), c: (0.6–0.7), d: (0.7–1.0). We encode the word "dad".

1. First symbol 'd': the range narrows from [0, 1) to [0.7, 1.0).

2. The sub-range for each symbol within [0.7, 1.0) is found using low = low + range × (low of symbol) and high = low + range × (high of symbol): a → [0.700, 0.820), b → [0.820, 0.880), c → [0.880, 0.910), d → [0.910, 1.000).

3. Second symbol 'a': the range narrows to [0.700, 0.820).

4. Third symbol 'd': subdividing [0.700, 0.820) by d's sub-range (0.7–1.0) gives low = 0.700 + 0.120 × 0.7 = 0.784 and high = 0.700 + 0.120 × 1.0 = 0.820, i.e. the final range [0.784, 0.820).

Any number lying within the final interval, e.g. 0.802, can now be transmitted as the arithmetic code (the "tag") for the string "dad".

Decoding: The receiver gets the tag 0.802 and repeatedly locates it within the symbol sub-ranges, rescaling after each decoded symbol using t* = (tag − low) / range:

1. 0.802 lies in [0.7, 1.0) → decoded symbol is 'd'.

2. Rescaling: t* = (0.802 − 0.7)/0.3 = 0.34, which lies in [0.0, 0.4) → decoded symbol is 'a'.

3. Rescaling again: t* = (0.34 − 0.0)/0.4 = 0.85, which lies in [0.7, 1.0) → decoded symbol is 'd'.

The decoded sequence is therefore "dad", matching the original message.

Disadvantage: The main disadvantage of arithmetic coding is that it tends to be slow: full-precision arithmetic coding requires at least one multiplication/division per symbol, and some implementations require up to two multiplications and two divisions per symbol. In practice, it is impractical to use arithmetic coding alone, but it is used as one of the entropy coding options within the JPEG standard.

Observations:

- Students will understand prefix and uniquely decodable codes and why they permit unambiguous, instantaneous decoding.
- Students will construct Shannon-Fano and Huffman codes for a given histogram and compute the code efficiency using entropy.
- Students will implement arithmetic coding to encode and decode a short symbol sequence and compare its performance with Huffman coding.

Questions:

1. Problems based on checking whether a given variable-length code is uniquely decodable and/or a prefix code.

2. Problems based on constructing Shannon-Fano and Huffman codes for a given image histogram and computing their efficiency and compression ratio.

3. Problems based on encoding and decoding a short symbol sequence using arithmetic coding, and comparing its average code length with Huffman coding for a skewed probability distribution.

LABORATORY 11

Image Compression – II

Aim:

To study lossy image compression techniques – gray-level run-length coding, block truncation coding, vector quantization, and transform coding (as used in the JPEG standard) – and to compute the resulting compression ratios.

Brief Theory:

Lossy compression methods are required to achieve high compression ratios with complex images. They provide an adjustable trade-off between image quality and degree of compression, allowing the compression algorithm to be customized to the application. With advanced methods, images can be compressed 10 to 20 times with virtually no visible information loss, and 30 to 50 times with only minimal degradation; newer techniques such as JPEG2000 can achieve reasonably good image quality at compression ratios as high as 100 to 200. Image enhancement and restoration techniques can be combined with lossy compression to improve the appearance of the decompressed image. In general, a higher compression ratio results in a poorer image, but the results are highly image dependent. Lossy compression can be performed in the spatial domain, the transform domain, or using hybrid methods that use both.

Gray-Level Run-Length Coding

The run-length coding (RLC) technique, which is lossless when applied directly, can also be used for lossy image compression by first reducing the number of gray levels (bit-plane reduction) and then applying standard RLC techniques. As with the lossless techniques, a preprocessing step improves the compression ratio further. For an 8 bits/pixel, 256 gray-level image, reducing the number of bits/pixel gives the following approximate compression ratios (without / with preprocessing): 7 bpp (128 levels) – 0.55 / 0.66; 6 bpp (64 levels) – 0.77 / 0.97; 5 bpp (32 levels) – 1.20 / 1.60; 4 bpp (16 levels) – 2.17 / 2.79; 3 bpp (8 levels) – 4.86 / 5.82; 2 bpp (4 levels) – 13.18 / 15.44; and 1 bpp (2 levels) – 44.46 / 44.46. Note that no actual compression occurs until the image is reduced to about 5 bits/pixel.

Block Truncation Coding (BTC)

Block Truncation Coding, first presented by Delp and Mitchell in 1979, is based on preserving the first and second statistical moments (the mean and variance) of the image. BTC works by dividing the image into small n × n blocks and then reducing the number of gray levels within each block to two, using a quantizer that adapts to the local block statistics. The two levels, a high value H and a low value L, are chosen so that the mean and variance of the gray levels within the block are preserved; each pixel is then compared with a threshold (typically the block mean) and assigned the high-level code if it is above the mean, or the low-level code if it is below the mean.

$H = m_b + \sigma_b \sqrt{[(n^2 - q)/q]}$

$L = m_b - \sigma_b \sqrt{[q/(n^2 - q)]}$

where the block size is n × n, m_b is the block mean, σ_b is the block standard deviation, and q is the number of pixel values in the block that are ≥ m_b. For n = 4, the 4×4 block is encoded with just four bytes: two bytes to store the levels H and L, and two bytes (16 bits) to store the bit string of 1's (pixel ≥ mean, coded H) and 0's (pixel < mean, coded L) for that block.

Example: For the 4×4 subimage with values [12,16,15,17; 13,16,17,17; 4,4,35,35; 42,42,12,12], the block mean is m_b = 19.3125 and the block standard deviation is σ_b ≈ 11.85. There are q = 4 pixel values greater than or equal to the mean, giving H ≈ 40 and L ≈ 13. Assigning a '0' to each pixel

below the mean and a '1' to each pixel at or above the mean gives the 16-bit bit string 0000 0000 1111 00, which along with H and L is stored for that block; on decompression, every '1' position is reconstructed as 40 and every '0' position as 13.

Basic BTC tends to produce images with blocky effects; these artefacts can be smoothed by post-processing with enhancement techniques such as median or averaging (lowpass) filters. BTC lends itself very well to parallel processing, since each block is coded completely independently. The multilevel BTC algorithm uses a 4-level quantizer and allows the block size to be varied; a larger block size gives higher compression but a corresponding decrease in image quality (e.g. 8×8, 16×16, or 32×32 blocks).

Vector Quantization (VQ)

Vector quantization is the process of mapping a vector that can take many values to a vector from a smaller (quantized) set of values. For image compression, the vector corresponds to a small subimage or block, e.g. a 4×4 block re-arranged row-by-row into a single 1-D vector. Information theory shows that better compression can be achieved with vector quantization than with scalar quantization (rounding or truncating individual pixel values), since VQ treats the entire block as a single entity and quantizes it using a codebook, a fixed set of representative vectors, coding each block by simply storing the index (address) of the closest matching vector in the codebook.

Example: For an 8-bit 256×256 image encoded with a codebook of 256 entries using 4×4 blocks, each block is stored as a single byte (its codebook index). This gives (256/4)×(256/4) = 4096 blocks, so if the codebook itself is not transmitted, the compression ratio is 16:1. If the 256×16-byte codebook (4096 bytes) is included along with the 4096 index bytes, the coded file requires 8192 bytes versus 65,536 bytes for the original image, giving a compression ratio of only 8:1, including the codebook cuts the compression ratio in half.

The codebook is generated by a training algorithm that determines a set of vectors best representing the blocks of the image, by minimizing an error criterion defined as the sum of the vector distances between the original and decompressed subimages. A codebook may be generic (reusable across similar images, so only its name needs to be stored) or designed specifically for a given image (giving better quality but requiring the codebook to be stored or transmitted). VQ is well suited to applications where images are compressed once but decompressed many times, such as images on a website, since decompression is simple and fast, though compression (codebook design) is comparatively complex and costly. As the codebook size is increased, image quality improves but the compression ratio decreases (e.g. 128 entries → ratio 11.49; 256 entries → ratio 7.93; 512 entries → ratio 5.09). VQ can be applied in either the spatial domain or a transform domain (e.g. DCT or wavelet domain).

Transform Coding (JPEG Standard)

Transform coding is a form of block coding performed in the transform domain: the image is divided into blocks (subimages), and a transform is calculated for each block. Although transforms such as the Fourier or Walsh-Hadamard transform can be used, the Discrete Cosine Transform (DCT) has been found to be optimal for most images and is used in the JPEG standard, while the newer JPEG2000 standard uses the wavelet transform, which provides even better compression. After the transform is computed, the resulting coefficients are quantized and coded. This approach is effective because the transform is very efficient at packing most of the image information (energy) into relatively few low-frequency coefficients, so that many of the high-frequency coefficients can be quantized to zero and effectively discarded, with the main goal of the transform being to pack as much information as possible into as few coefficients as possible before quantization and coding.

Observations:

- Students will implement lossy run-length coding via gray-level (bit-plane) reduction and observe the trade-off between bits/pixel, compression ratio, and visual quality.
- Students will implement Block Truncation Coding and observe the resulting blocky artefacts and their reduction using post-filtering.
- Students will implement Vector Quantization with a codebook and study how codebook size affects compression ratio and reconstructed image quality.
- Students will study transform (DCT-based) coding as used in the JPEG standard.

Questions:

1. Problems based on lossy run-length coding by gray-level reduction, with and without preprocessing.

2. Problems based on Block Truncation Coding: compute the high and low values (H, L) and the bit-map for a given subimage block.

3. Problems based on Vector Quantization: design/apply a codebook and compute the resulting compression ratio with and without storing the codebook.

4. Problems based on comparing compression ratio and image quality for different transform-domain (DCT / wavelet) based lossy compression schemes.

LABORATORY 12

Image Representation and Description

Aim:

To study various image description techniques.

Brief Theory:

A segmented region can be represented by boundary pixels or internal pixels. When shape is important, a boundary (external) representation is used. When colour or texture is important, an internal representation is used. The description of a region is based on its representation, for example a boundary can be described by its length. The features selected as descriptors are usually required to be as insensitive as possible to variations in scale, translation, and rotation, that is the features should be scale, translation and rotation invariant.

The chain code is a slope intrinsic representation of a shape. The chain code of a digitized curve represents the sequences of direction changes between adjacent points on the curve. If the nth point of the curve is at grid position (i, j), then the chain element, or link, corresponding to the change in position from nth point to the $(n+1)^{th}$ point.

The purpose in constructing the medial surface or medial axis (skeleton, deformation retract) of an object is to obtain an reduced representation of the object that is easier to analyze. *Skeletonization* of a digitized object is performed by careful erosion of the object's pixels, layer by layer, while preserving the object's topological and geometrical properties (essentially, a voxel is eroded (removed) only if its removal does not induce a local change in topology, e.g. breaking the object in two parts, creating a hole or cavity). Conceptually, the medial erosion of a 3D object can be viewed as first generating a medial surface and, if the object has no embedded cavities, eroding this surface further to a union of digitized curves referred to as the medial axis (MA). (In practice MA algorithms generate the medial axis with no intermediate generation of a medial surface.)

Observations:

- The students will learn different descriptors for recognition.
- The students will study medial-axis generation of an object.
- The student will apply these concepts for solving various recognition tasks.

Questions:

1. Problems based on boundary descriptors.
2. Problems based on region descriptors.
3. Problems based on thinning.

LABRATORY 13

Multiresolution analysis and wavelets

Aim:

To study about multiresolution and wavelets.

Brief Theory:

Unlike the Fourier transform, which decomposes a signal to a sum of sinusoids, the wavelet transform decomposes a signal (image) to small waves of varying frequency and limited duration. The advantage is that we also know when (where) the frequency appear. Many applications in image compression, transmission, and analysis.

Small objects are viewed at high resolutions. Large objects require only a coarse resolution. Images have locally varying statistics resulting in combinations of edges, abrupt features and homogeneous regions. An image pyramid is a collection of images at decreasing resolution levels. At each reduced resolution level, it have a filtered and downsampled image.

In subband coding, an image is decomposed to a set of bandlimited components (subbands). The decomposition is carried by filtering and downsampling. If the filters are properly selected the image may be reconstructed without error by filtering and upsampling. The goal of subband coding is to select the analysis and synthesis filters in order to have perfect reconstruction of the signal. It may be shown that the synthesis filters should be modulated versions of the analysis filters with one (and only one) synthesis filter being sign reversed of an analysis filter.

Observations:

- The students will develop an understanding for wavelets.
- The students will explore some of multiresolution techniques.
- The student will apply wavelet transform for vaious image processing tasks.

Questions:

1. Problems based on image pyramids.
2. Problems based on subband coding.
3. Problems based on application of wavelets.

LABRATORY 14

Syllabus:

Questions will be given from the topics covered throughout labs.

Structure:

1. **Two** problems will be given for execution.
2. Time**: 2:30 hrs**.

Instructions:

The students should follow the instructions given below.

1. You are not allowed to use mobile phones during the lab examination.
2. Use of internet is prohibited during the lab examination.
3. After completion of lab examination, you need to submit your programs through email. The respective *email ID*s will be provided during the exam.
4. You should have your ID cards for verification.